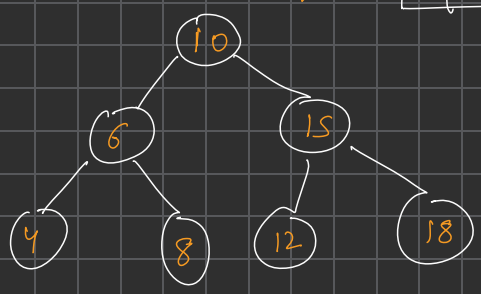
B.S.T : Binary Search Tree:

| | | | | | |
|---|---|---|---|---|---|
| 3 | 5 | 1 | 8 | 2 | 7 |

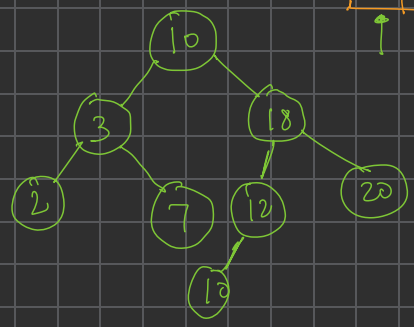
18

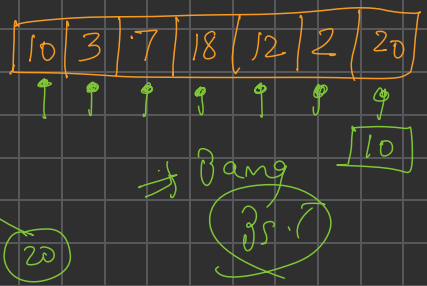
| | | | | | |
|---|---|---|---|---|---|
| | | | | | |



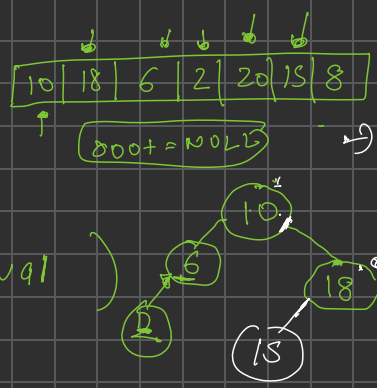
(1) → Node <= Left side
other Right

(2) Node < left side
other Right



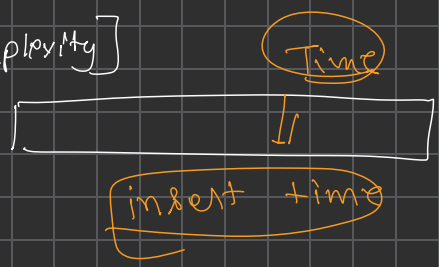



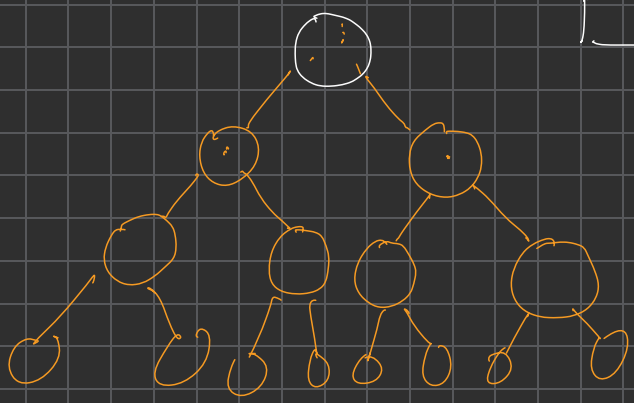

```
class Node{
  public:
    int data;
    Node *left, *right;
}

Node* insert(Node *root, val)
{
    if (!root)
    {   return new Node(val);
    }

    if (val < root->data)
    {
    root->left = insert(root->left, val);
    }
    else
    {
    root->right = insert(root->right, val);
```

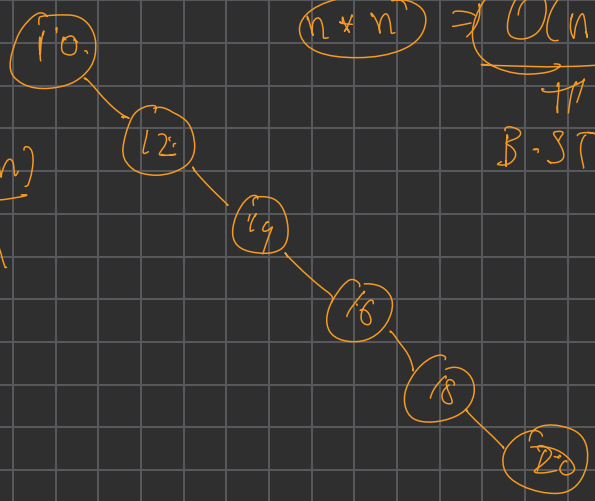


```
    return root;
}


int main()
{
    Node * root = NULL;
    for (i=0; i<n; i++)
    {
        root = insert(root, arr[i]);
    }
}
```

[Q.8 T: Time Complexity]






$n * n \Rightarrow O(n^2)$

$\Downarrow$

B.ST

insert : $O(n)$

Search : $n$

(1) Left : Righmest nd

Node* find (Node *root) {root

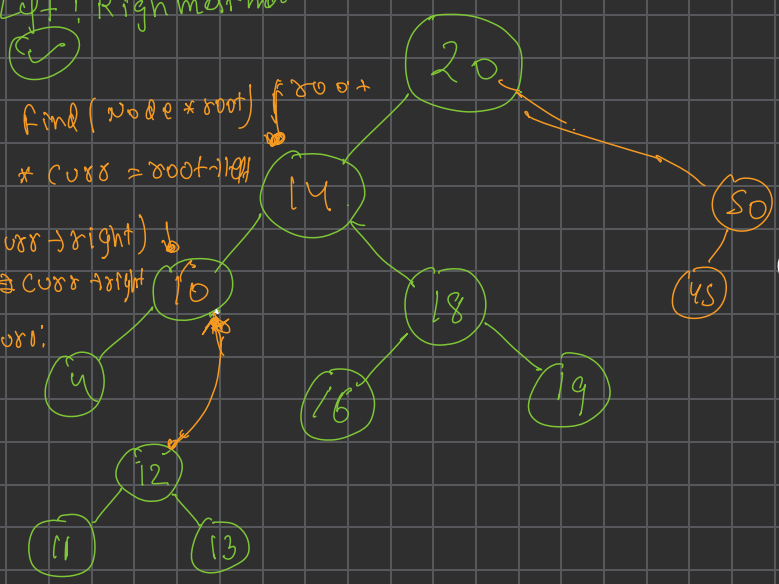
{ Node * curr = root->left

while(curr->right) {

curr = curr->right

return curr;

}




delete

15 leaf n..

so

(1) Leaf Nod

→ delete it

→ return

(2) Non leaf

```
Node *    delete ( Node *root , int val )
{
    if( !root)
        retur NULL;

    if ( val < root -> data)
    {
    root->left= delete (root -> left, val );
    }
    else if ( val > root -> data)
    {
        root -> right = delete (root-> right , val);
    }
    else
    {
        // leaf node;
        if( !root -> left && !root -> right)
        {
            delete root;
            return NULL;
        }
        else if ( !root -> left )
        {
            Node *temp = root -> right;
            delete root ;
            return temp;
        }
```

```
else if (!root->right)
{
    Node *temp = root->left
    delete root;
    return;
}
else{
    Node *pred = find(root);
    root->data = pred->data;
    root->left = delete(root->left, pred->data)
}

r
```

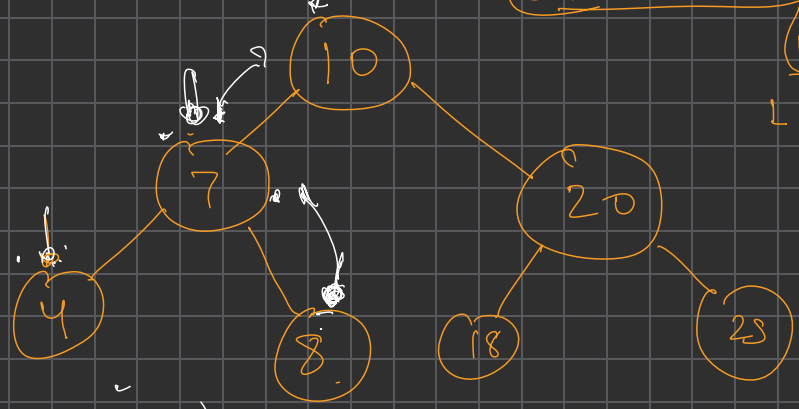
result = 8

k = 3 2 1 0

Extra space

k = 3

L N R




```
void find (Node *root, int &k, int &result)
{
    if (!root || k <= 0)
        return;

    find (root->left, k, result);
    k--;
    if (k==0)
    {
        result = root->data;
        return;
    }
```

```
find(root->right, k, result);

}
```